The Relay: an evidence-first RAG assistant over three papers

1 What I built and why

A Python assistant that answers only from three pinned arXiv papers (Attention v7, Moshi v1, Dynamic Chunking v2). Every answer carries a paper, a page, a section and a verbatim quote; the quote is checked by a tool call against the cited chunk *before* the answer is shown, and the system refuses rather than guess. Pipeline: analyse the question (is the corpus enough, which passages) -> BM25 retrieval over headings plus body (349 chunks, 350 tokens, 60 overlap) -> draft with JSON citations -> `verify_quote` -> at most two repairs -> refuse.

The decision that shaped everything else: **admission control on tokens, not retries on 429.** The free tier allows 30 requests per minute but only 8,000 tokens per minute. A measured evaluation answer costs 2,406 tokens (1,662 in, 745 out), so 8,000 tokens per minute buys about three answers a minute where 30 requests per minute would allow thirty: the token ceiling binds about nine times earlier. The scheduler therefore estimates the cost before sending and refuses with a retry hint when a request cannot finish inside its deadline. Retries exist, but only for genuine failures.

2 The real service and my stand-in

Documented free-tier limits for `openai/gpt-oss-120b` (console.groq.com/docs/rate-limits, read 9 Nov 2026): 30 RPM, 1,000 RPD, 8,000 TPM, 200,000 TPD, 429 with `retry-after`. On 11 Nov 2026 `relay probe-limits` measured the live service with a real key. The 429 body states `Limit 30` on requests per minute - 20 of 32 concurrent calls fit because ten earlier calls were still inside the window - and the `x-ratelimit-limit-tokens` / `-requests` headers state 8,000 TPM and 1,000 RPD. A rejection is HTTP 429 with `retry-after: 2.0`, and capacity returned after 3.0 s of polling. Ten small prompts that produced visible output measured first token at p50 525 ms, p95 714 ms. Tokens per day stays *not measured*: proving it would spend the entire day's budget.

The first attempt was wrong: at `max_tokens=6` every token went to reasoning, so each call returned no content and a null TTFT that the summary averaged as 0 ms. The sample now allows 64 tokens, and the TPM probe runs before the request burst so a request-rate 429 cannot pass for a token-rate one.

The stand-in (`services/standin`) speaks the same OpenAI SSE protocol with real counters for those limits, so client code is identical for both sources. `relay calibrate` has now fitted it to the probe: `services/standin/params.json` carries the measured TTFT (525 ms p50, 714 ms p90) and per-token cost, and `/health` reports `calibrated: true`. Where it cannot match: no cold start, TTFT independent of prefill length, provider queue depth approximated as a linear slowdown. One caveat on the load numbers: those scenarios were run before this fit, against the stand-in's 190 ms default, so their absolute latencies understate the real service; the scheduling outcomes (who waits, who is shed) do not depend on that constant. Every CSV row carries `source`, and every load number here is labelled stand-in.

Table 1 - limits: documented versus observed

Limit	Documented	Observed	How found	Confidence
Requests per minute	30	30 (429 body; 20 of 32 calls fit)	32 concurrent minimal calls	measured
Tokens per minute	8000	8000	`x-ratelimit-limit-tokens` header	header
Requests per day	1000	1000	`x-ratelimit-limit-requests` header	header
Tokens per day	200000	not measured	would cost 200k tokens to reach	documented
Requests in flight	-	not measured	ramped concurrency until refusal	not measured
What happens when a limit is hit	-	HTTP 429, retry-after [2.0] s	captured 429 response	measured
Time until recovery	-	3.0 s	polled every 2 s	measured

3 Results

493 requests: 60 evaluation (A on the real service, B and C on the stand-in, 20 questions each) and 433 load, 173,064 tokens in, 24,938 out, 40 retries, 157 of them answered. Tables 1-3 are inserted by `report/build_pdf.py` straight from `report/TABLES.md`, which only `relay tables --write` writes, so the numbers below and `results/results.csv` cannot drift apart the way they did in the last draft. Latency covers **successful requests only**; refusals are counted in 2b, because a 7 ms refusal is not a 7 ms answer.

Table 2a - latency by scenario (ms), successful requests

Scenario	Source	Requests	Answered	Duration (s)	Offered/min	TTFT p50	TTFT p95	TTFT p99	Total p50	Total p95	Total p99
burst	stand-in	30	5	3.0	600.0	162	328	359	1791	2419	2501
daily_exhausted	stand-in	12	0	11.0	60.0	not measured	not measured	not measured	not measured	not measured	not measured
degraded	stand-in	30	19	146.3	12.0	166	600	1012	1607	27327	30549
faulty	stand-in	40	12	92.0	30.0	142	874	937	1408	2342	2394
limit_halved	stand-in	22	11	69.0	25.0	93	325	358	25420	32859	33817
overload	stand-in	45	6	44.0	60.0	172	348	351	1574	1983	2032
steady	stand-in	14	14	196.6	4.0	175	744	1049	1583	2120	2164
sustained	stand-in	240	30	239.0	60.0	166	757	1035	1379	1834	1970

Table 2b - reliability, retries, queue and cost

Scenario	Succeeded	Rate-limited	Timed out or given up	Retries/request	Peak queue	Tokens out/answer	Cost/1000 answers
burst	16.7%	53.3%	30.0%	0.27	19	88	$0.00 (free tier)
daily_exhausted	0.0%	100.0%	0.0%	0.00	1	not measured	$0.00 (free tier)
degraded	63.3%	30.0%	6.7%	0.27	3	64	$0.00 (free tier)
faulty	30.0%	62.5%	7.5%	0.20	1	67	$0.00 (free tier)
limit_halved	50.0%	0.0%	50.0%	0.00	5	70	$0.00 (free tier)
overload	13.3%	86.7%	0.0%	0.18	1	84	$0.00 (free tier)
steady	100.0%	0.0%	0.0%	0.00	1	75	$0.00 (free tier)
sustained	12.5%	87.5%	0.0%	0.03	1	65	$0.00 (free tier)

The number I could not have guessed: **30 requests per minute is not 30 answers per minute.** At 4/min everything succeeds (14/14, total p50 1,583 ms). Held at 60/min for four minutes, 30 of 240 succeed (12.5%). And the queue is real rather than decorative: burst offers 30 calls at once against 4 in-flight slots, so 19 requests wait behind them — 16 are then refused on the token budget, and 9 more spend their 30 s deadline in the queue and are recorded as `timeout`, not as a fast no. That is the honest cost of queueing: waiting converts instant refusals into slow failures, and only 5 of 30 get an answer.

4 Where the time goes

Steady state: 1,583 ms p50, of which 175 ms to the first token (11.1%) and 1,408 ms after it (88.9%), with 0 ms queue wait and 0 ms retrying.

The slowest successful request in the CSV is **`f0bc6f94fd14`**, a real evaluation answer at 59,018 ms: **limit wait 51,863 ms (87.9%) + working 7,152 ms (12.1%) + queue 0 + retrying 0**, first token at 994 ms, and two repairs after the draft's quotes failed verification. The provider's minute window was full, so most of that time was our own limiter holding a reservation; the model worked for 7 s across four calls. The stand-in shows the same shape: `065eb61738d6` (limit-halved, 34,056 ms) is 29,499 ms of limit wait (86.6%) plus 1,337 ms of work (3.9%) on 328 tokens of context. Not model slowness, not retries, not queueing.

That split did not exist until this was investigated: the original trace of a similar request showed 50,411 ms total with 0 ms queued and 961 ms working, i.e. 49 s unaccounted. The gap was time blocked inside `limiter.acquire`, which nothing was timing. `limit_wait_ms` now records it, and every request in this draft was run with that instrumentation. The split is structural: when the ladder engages, the first call is skipped entirely, so what the person waits for is budget, then one draft plus verification.

5 When a limit is hit

Per-minute: the provider returns 429 with `retry-after`. If the wait fits the deadline the limiter sleeps and retries (at most twice); otherwise it refuses at once, quoting the wait. **Who is delayed:** with 4 in-flight slots, whoever is behind the queue. Burst shows the trade-off plainly - 19 requests queue, 16 are shed on token budget, 9 exhaust the 30 s deadline while waiting (recorded as `timeout`, because the wait was ours, not the provider's), and 5 are answered. Halving both windows is the only scenario where waiting mostly works: 11 of 22 answered (50.0%), the other 11 timed out after waiting, and **0 were refused instantly**, so the halved run is queue-and-deadline, not shedding. Per-day: no wait inside a request restores a daily budget, so it rejects before sending - 0 retries, 0 ms, 100% `rate_limited`.

Degradation is now measured, not asserted. With 65% of the minute already spent, the ladder engages by itself: levels 0/1/2 appear as 3/16/11 of 30 requests, average context falls from 578 tokens (steady) to 478, and average model calls from 1.64 to 1.20 - it skips the analysis call, then trims context to one or two passages and drops the repair loop. In the halved run 18 of 22 requests ran at level 2 and average context fell to 333 tokens. Nobody in these runs received a *weaker answer* rather than a refusal, because the bucket empties before the trimmed path can complete; what is measured is that the system sheds work before it sheds correctness.

Retrieval: quality versus cost

Table 3 - retrieval settings, quality and cost (A real, B and C stand-in; automatic grades)

Setting	Source	Piece size	Fetched	Re-rank	Questions	Correct and cited	Wrong	Refused correctly	Refused when answerable	Quotes verified	Tokens out/answer	TTFT p50	TTFT p95	Total p50	Total p95
A	real	350/60	4	none	20	15.0%	25.0%	30.0%	30.0%	100.0%	745	912	2015	3761	58332
B	stand-in	700/100	4	none	20	20.0%	80.0%	0.0%	0.0%	not measured	80	168	585	1647	2455
C	stand-in	350/60	16	cross_encoder	20	25.0%	75.0%	0.0%	0.0%	not measured	81	221	824	1723	2505

Only setting A has been run against the real service, and that row is the one that means anything: of 20 questions, **3 were answered and cited in the gold region, 5 were answered from the wrong passage, 6 were refused although the corpus contains the answer, and 6 were refused correctly**. Every quote the system did ship was matched exactly by `verify_quote` (100%), so the failure is not a bad quote - it is a system that too often says it cannot answer (30%) or answers from the wrong section (25%). B and C are stand-in rows; their 0% refusals come from the simulator's `answerable = bool(question)`, and their 20%/25% correct rates are a scripted-citation artefact, not a verdict on chunking or re-ranking. The table can therefore report A's real cost (745 output tokens per answer, TTFT 912 ms p50, total 3.8 s p50) but it cannot rank A against B and C on quality: that needs two more real runs, and the 200,000-token daily budget did not allow them on the same day. What the stand-in rows still say is cost: B's larger chunks and C's re-ranking both add TTFT over A. When the limit is close, retrieval is the first thing squeezed: the ladder cuts context to one passage (328 tokens in `065eb61738d6`), so a near-limit answer is cheaper and shallower.

How this library will mislead a naive retriever

1. **Appendix gravity.** Moshi is 66 pages and supplies 172 of 349 chunks; appendices repeat "LibriSpeech" and "WER" densely. `q06` (streaming WER, section 5.7) returns appendix tables first - the citation looks right and points at the wrong section.
2. **Cross-paper term collision.** `q11` asks for batch size and training steps (Attention section 5.1); the top four fill with Moshi and H-Net training sections that use exactly those words. This is why gold is a page-and-section region, not a chunk id.
3. **Heading versus body.** `q01` asks for the key dimension in section 3.2, but the body repeats "attention" and "heads" everywhere, so a body-only index returns generic 3.2.1 text. Indexing headings with each chunk lifted that passage into the top four.

6 Retries

Full-jitter exponential backoff (base 400 ms, cap 8 s), three attempts per call, a shared retry budget, and a rule that the delay is never shorter than the provider's `retry-after`. The breaker counts only 5xx and timeouts: a 429 means the provider is healthy and busy.

Measured: 40 retries across 493 requests (0.08 each), all of them in the five fault-bearing load scenarios and none when limits are halved, where the limiter refuses or waits instead of retrying. 38 rows were retried; **2 of them succeeded**. That ratio is the argument for admission control: retrying mostly bought latency, not answers. **A controlled experiment where retrying is worse** (scripted provider, so it says nothing about Groq's fault rate): one 500 fails in 1 attempt with `max_attempts=1`, but in 3 attempts with `max_attempts=3`, taking more than 2.5x the wall clock - all of it backoff. Same outcome, later. A 429 whose `retry-after` (55 s) exceeds the 30 s deadline is refused in under 0.5 s rather than slept on, which is the same lesson applied.

That policy came from a failure: the first evaluation counted 429s as breaker failures, opened the circuit for 20 s and turned 13 requests into instant `error` rows. The fix is decision D6 in `docs/DECISIONS.md`.

7 What the person sees

When the daily budget is gone, this is the reply, verbatim:

> *I can't answer this one right now: today's free-tier model budget is spent. The account allows 200,000 tokens and 1,000 requests per day, shared with every other run, and it resets at the start of the next UTC day. No partial or guessed answer was sent. Retry after the reset, or narrow the question so it needs fewer passages.*

At the per-minute limit the message names the wait ("It frees up in about 55 seconds. Nothing was lost - retry then.") and repeats the promise: no shorter, unverified answer was sent to stay fast.

8 What I am unsure about

Least certain: that `retry-after` estimates when capacity returns. It is a hint from one window; if it underestimates, the limiter sleeps, retries and fails anyway - strictly worse than refusing, and exactly what the 50.0% timeout share in the halved run looks like: those requests waited the full deadline and were told no at the end of it. What breaks first: multi-paper synthesis, where nothing forces a cross-paper answer to cite more than one paper. Also unresolved: tokens per day is unmeasured (reaching it would spend the whole budget), the real quality sample is 20 questions from one model, so 3/20 carries a wide interval, and the stand-in's fault rates (15% 500s, 5% timeouts, 10% truncated streams in `faulty`) are synthetic - they prove the handling, not the frequency. Not built: multi-provider failover and an answer cache; neither changes the token ceiling. What I learned: a latency budget is set by the slowest *mandatory* step, and measurement can hide the biggest component - 29.5 s of a 34 s request, and 51.9 s of a 59 s one, were invisible until the limiter was instrumented.

Your feedback on Vaani

Checked against the live pages on 10 Nov 2026, not from memory.

Standout: "Zero dead air" and the screening section - Vaani answers the carrier prompt and stays on the line, with ~98% of screening calls handled under 15 s - are framed as outcomes, not model features. That is the right way to sell a voice agent, and it is the unglamorous work that decides whether outbound campaigns survive.

Confusing: the hero's three counters render as `0M+`, `0.0%` and `<0ms` with labels ("Response accuracy", "Sub-500ms... P-90 latency") that do not sit under the numbers they describe, so the most prominent evidence on the page reads as broken. The body separately claims "p90 latency of ~450ms", which is credible but unsourced. And the CTA says "Start free on Vaani's Platform" while `/pricing` offers only pay-as-you-go (prepaid wallet, ₹1/min platform fee, 2 concurrent calls) and Enterprise - so "free" is not a plan a reader can find.

One change: publish the measurement method beside the latency claim (audio conditions, line type, sample size) and make the hero counters carry their denominators. For a latency-led product the method is part of the pitch. *(Web pages only; I have not run a live call or spoken to the team, so the accuracy counter's provenance stays unverified.)*

How I worked with the assistant

Three prompts that mattered: "plan before coding, with a gate per stage"; "measure the limits before designing the limiter"; and "every table must regenerate from `results.csv`". One thing it got wrong: grading evaluation rows after the pipeline had written them, producing duplicate, partly ungraded rows - caught by counting rows against questions. Two things it got wrong later: the breaker treating 429s as outages (13 instant errors), and a 49 s unaccounted gap in a 50 s request that only appeared when one request was split instead of summarised. One overrule: it proposed a vector store for 349 chunks; BM25 over headings stayed. What is distinctly mine: the token-admission decision, verify-before-reveal, labelling every stand-in row, and this report's refusal to fill unmeasured cells.